ShopKart RAG Assistant

Answer Sheet: complete code, knowledge base and full explanation

Companion to the ShopKart RAG Project Assignment

About the data. The assignment only listed *topics* for the delivery, return and payment files, so the numbers in those three files below (3 to 5 days, 7-day returns, 99 INR express and so on) are fictional values written for this answer sheet. If you write your own files, your numbers will differ and your answers will change to match. The code stays the same.

1. How the system works

A normal chatbot answers from what the language model memorised in training. It can be out of date, and it will happily invent a price. **RAG (Retrieval-Augmented Generation)** fixes this by looking the facts up in your own documents first, then asking the model to answer *only* from what was found.

The system has two phases:

- **Indexing (once, at start-up):** load the four files, cut them into chunks, turn every chunk into a vector (a list of 384 numbers that captures its meaning), and keep the vectors in memory.
- **Answering (every question):** turn the question into a vector the same way, find the chunks whose vectors are closest, paste those chunks into a prompt, and let the LLM write a grounded answer.

Step	Code	Assignment task
Load the four files	load_documents()	Task 1
Cut into overlapping chunks, keep filename	chunk(), build_records()	Task 2
Embed every chunk (normalized)	embedder.encode(...)	Task 3
Embed question, rank chunks by cosine similarity	retrieve()	Task 4
Build context, ask the LLM, show sources	answer()	Task 5
Say "not found" when the knowledge base has no answer	threshold + prompt rule	Task 6

2. Project setup

```
shopkart/
    kb/
        products.txt
        delivery_policy.txt
        return_refund_policy.txt
        payment_policy.txt
    rag_shopkart.py
    .env                  <- contains: GROQ_API_KEY=your_key_here
```

Install the libraries, then run:

```
pip install sentence-transformers numpy groq python-dotenv

python rag_shopkart.py            # interactive chat
python rag_shopkart.py --test     # runs all 23 test questions
```

The first run downloads the all-MiniLM-L6-v2 model (roughly 90 MB), so it is slow once and fast afterwards. The assignment mentions the OpenAI API; this answer sheet uses Groq because that is the key you have. Only the client creation and the `chat.completions.create` call would change for OpenAI.

3. Knowledge base files (Task 1 data, 10 marks)

These are the four files to place inside the `kb/` folder. Line breaks are added here only so they fit the page; in the real files each paragraph is a single line.

products.txt

```
SHOPKART PRODUCT CATALOG

PRODUCT 1
Product ID: SK101
Name: Samsung Galaxy M35
Category: Smartphone
Price: 19,999 INR
Brand: Samsung
RAM: 8GB
Storage: 128GB
Battery: 6000mAh
Display: Super AMOLED
Availability: In Stock
Warranty: 1 Year

PRODUCT 2
Product ID: SK102
Name: Redmi Note 14
Category: Smartphone
Price: 17,999 INR
Brand: Xiaomi
RAM: 8GB
Storage: 128GB
Battery: 5500mAh
Display: AMOLED
Availability: In Stock
Warranty: 1 Year

PRODUCT 3
Product ID: SK103
Name: HP Pavilion Laptop
Category: Laptop
Price: 59,999 INR
Brand: HP
RAM: 16GB
Storage: 512GB SSD
Processor: Intel Core i5
Availability: In Stock
Warranty: 1 Year

PRODUCT 4
Product ID: SK104
Name: boAt Airdopes 141
Category: Earbuds
Price: 1,299 INR
Brand: boAt
Battery Life: 42 Hours
Connectivity: Bluetooth
Availability: Out of Stock
Warranty: 1 Year
```

delivery_policy.txt

```
SHOPKART DELIVERY POLICY

Delivery charges: Standard delivery is free on orders of 499 INR and above. Orders below 499
INR are charged a delivery fee of 40 INR.

Standard delivery time: Standard delivery takes 3 to 5 business days after the order is
shipped.

Express delivery: Express delivery is available in metro cities only. It takes 1 to 2 business
days and costs an extra 99 INR per order.

Tracking orders: Once an order is shipped, ShopKart sends a tracking link by SMS and email.
Customers can also track the order under the My Orders section of the ShopKart website or app.

Delivery locations: ShopKart delivers to all serviceable PIN codes across India. International
delivery is not available.

Delayed delivery policy: If an order is not delivered within 2 business days after the
promised delivery date, the customer can contact ShopKart support. If the delay is more than 7
days, the customer can cancel the order and receive a full refund. The express delivery fee is
refunded if an express order is late.
```

return_refund_policy.txt

```
SHOPKART RETURN AND REFUND POLICY

Return eligibility: A product can be returned if it is unused, in its original packaging, and
returned with all accessories, tags and the invoice.

Return window: Customers can request a return within 7 days of delivery.

Refund processing time: Refunds are processed within 5 to 7 business days after the returned
product passes the quality check.

Replacement policy: A replacement is offered for defective or wrong items if the request is
made within 7 days of delivery. If the replacement item is out of stock, ShopKart gives a full
refund instead.

Damaged product procedure: If a product arrives damaged, report it within 48 hours of delivery
under My Orders and upload photos or an unboxing video. ShopKart arranges a free pickup and
offers a replacement or a full refund.

Non-returnable products: The following cannot be returned: earbuds and other in-ear audio
products once the seal is opened, innerwear, gift cards, software and digital products, and
any product with a missing or tampered serial number.
```

payment_policy.txt

```
SHOPKART PAYMENT POLICY

UPI: ShopKart accepts UPI payments through Google Pay, PhonePe, Paytm and BHIM. UPI payments
are confirmed instantly.

Credit and debit cards: Visa, Mastercard and RuPay credit and debit cards are accepted. Every
card payment is verified with a one-time password (OTP).

Cash on delivery: Cash on delivery is available for orders up to 10,000 INR in serviceable PIN
codes. A cash on delivery fee of 30 INR applies.

EMI: No-cost EMI is available on select credit cards for orders above 3,000 INR, with 3, 6 or
9 month plans. Debit card EMI is available from select banks.

Payment failures: If money is deducted but the order is not confirmed, the amount is refunded
automatically to the original payment method within 5 to 7 business days. The customer can
retry the payment, and should contact support if the refund does not arrive after 7 business
days.

Refund payment methods: Refunds go back to the original payment method. For cash on delivery
orders, the refund is sent to the customer's bank account by bank transfer or UPI.
```

Why the files are written this way

- **Every fact sits next to its label.** Writing "Delivery charges: ..." or "Return window: ..." means the chunk containing the answer also contains the words a customer would use. That is what the embedding model matches on.
- **One topic per paragraph.** Short, self-contained paragraphs mean a chunk rarely mixes unrelated policies.
- **Concrete numbers.** "3 to 5 business days" can be checked; "a few days" cannot. This makes the test questions gradeable.
- **Deliberate gaps.** There is no iPhone, no refrigerator and no CEO phone number anywhere. That is what the hallucination tests (questions 21 to 23) rely on.

4. Complete code, task by task

The pieces below are the real file `rag_shopkart.py` in order. Put them together from top to bottom to get the full program.

Setup: imports and settings

```
import os
import sys
from pathlib import Path

import numpy as np
from dotenv import load_dotenv
from groq import Groq
from sentence_transformers import SentenceTransformer

load_dotenv()  # reads GROQ_API_KEY from the .env file

KB_DIR = Path("kb")
KB_FILES = [
    "products.txt",
    "delivery_policy.txt",
    "return_refund_policy.txt",
    "payment_policy.txt",
]
CHUNK_SIZE = 50    # words per chunk
OVERLAP = 10       # words shared between neighbouring chunks
TOP_K = 4          # chunks sent to the LLM
MIN_SCORE = 0.25   # similarity threshold (below this = "not found")
LLM_MODEL = "llama-3.3-70b-versatile"
NOT_FOUND = ("I'm sorry, I couldn't find that information "
             "in the ShopKart knowledge base.")

_client = None


def get_client():
    """Create the Groq client the first time it is needed."""
    global _client
    if _client is None:
        _client = Groq(api_key=os.environ["GROQ_API_KEY"])
    return _client
```

All tunable numbers live at the top so you can experiment without hunting through the code. `load_dotenv()` copies the key from `.env` into the environment, so the key never appears in the source. The Groq client is created lazily in `get_client()`, which means the program can load and index the documents even before a key is set.

Task 1: load_documents() (10 marks, multiple document loading)

```
def load_documents(folder=KB_DIR):
    """Read the four .txt files. Returns [{'source': filename, 'text': str}]."""
    documents = []
    for name in KB_FILES:
        path = Path(folder) / name
        text = path.read_text(encoding="utf-8")
        documents.append({"source": name, "text": text})
    return documents
```

- `Path(folder) / name` builds the file path in a way that works on Windows, macOS and Linux.
- `encoding="utf-8"` avoids errors on symbols such as the rupee sign.
- Each document is a small dictionary, `{'source': 'products.txt', 'text': '...'}`. Keeping the filename with the text from the very first step is what makes source citations possible later.
- If a file is missing you get a clear `FileNotFoundError` naming it, which is better than silently building a knowledge base with a hole in it.

Task 2: chunk() (10 marks)

```
def chunk(text, size=50, overlap=10):
    """Split text into word-based chunks of `size` words, sharing `overlap`."""
    if overlap >= size:
        raise ValueError("overlap must be smaller than size")
    words = text.split()
    step = size - overlap
    chunks = []
    for start in range(0, len(words), step):
        chunks.append(" ".join(words[start:start + size]))
        if start + size >= len(words):   # this window already reached the end
            break
    return chunks


def build_records(documents):
    """Chunk every document and keep the filename as metadata."""
    records = []
    for doc in documents:
        for i, piece in enumerate(chunk(doc["text"], CHUNK_SIZE, OVERLAP)):
            records.append({"source": doc["source"], "chunk_id": i, "text": piece})
    return records
```

Why chunk at all? A whole policy file is too coarse: one vector for the entire delivery policy cannot tell you apart "express delivery" from "tracking". Small chunks give each fact its own vector, and they keep the prompt short.

How the loop works. With size=50 and overlap=10 the window moves forward by `step = 50 - 10 = 40` words each time:

```
words[0:50]     -> chunk 0
words[40:90]    -> chunk 1      (words 40..49 appear in both chunk 0 and chunk 1)
words[80:130]   -> chunk 2      (words 80..89 appear in both chunk 1 and chunk 2)
```

Why overlap? A fact that falls exactly on a boundary would otherwise be cut in half. The 10 shared words give it a second chance to appear whole in one of the two chunks. The `if start + size >= len(words): break` line stops the loop once a window has reached the end, so you do not get a tiny extra chunk made only of words that were already included.

build_records() loops over the documents and creates one record per chunk with the filename as metadata (Task 2 requirement):

```
{'source': 'products.txt', 'chunk_id': 1, 'text': 'Redmi Note 14 Category: Smartphone ...'}
```

With the four files above this produces **17 chunks**:

File	Chunks	What is inside
products.txt	3	#0 Samsung Galaxy M35 plus the start of Redmi; #1 Redmi Note 14 plus most of the HP laptop; #2 end of HP plus boAt Airdopes

File	Chunks	What is inside
delivery_policy.txt	5	charges, standard time, express, tracking, locations, delays
return_refund_policy.txt	4	eligibility, window, refund time, replacement, damaged, non-returnable
payment_policy.txt	5	UPI, cards, COD, EMI, failures, refund methods

Trade-off to know about. Word-count chunking is simple but it does not respect product boundaries, so a chunk can contain the end of one product and the start of the next. The overlap, the product name repeating in neighbouring chunks, and sending the top 4 chunks (not just 1) to the LLM make this safe for a catalog this small. Section 6 shows a better way to chunk the catalog (one chunk per product).

Task 3: embeddings (15 marks)

```
embedder = SentenceTransformer("all-MiniLM-L6-v2")

records = build_records(load_documents())
vectors = embedder.encode(
    [r["text"] for r in records],
    normalize_embeddings=True,    # unit-length vectors
)
print(f"Loaded {len(records)} chunks, vectors shape = {vectors.shape}")
```

- **An embedding** is a list of 384 numbers produced by the model. Texts with similar meaning get vectors that point in similar directions, even when they share no words ("cost" and "price").
- **normalize_embeddings=True** scales every vector to length 1. This is what lets us use a plain dot product as cosine similarity in Task 4. It is the single most important flag in the project.
- `vectors` is a NumPy array with one row per chunk. The printed line should read `Loaded 17 chunks, vectors shape = (17, 384)`: 17 chunks, 384 numbers each.
- This block runs once when the program starts. Questions only need to embed the question, not the whole catalog again.

Task 4: retrieve() (20 marks, the biggest part)

```
def retrieve(question, top_k=TOP_K, min_score=MIN_SCORE):
    """Return the top_k most similar chunks (with score) above min_score."""
    q_vec = embedder.encode([question], normalize_embeddings=True)[0]
    scores = vectors @ q_vec                    # cosine similarity per chunk
    best = np.argsort(scores)[::-1][:top_k]     # indices, highest score first
    return [
        {**records[i], "score": float(scores[i])}
        for i in best
        if scores[i] >= min_score
    ]
```

Cosine similarity measures the angle between two vectors: 1.0 means the same direction (same meaning), around 0 means unrelated. The formula is *dot(a, b) / (|a| x |b|)*. Because both vectors were normalized to length 1, the bottom part is 1 and cosine similarity is simply *dot(a, b)*.

- `vectors @ q_vec` does all 17 dot products in one line and gives an array of 17 scores.
- `np.argsort(scores)` sorts ascending, so `[::-1]` reverses it (best first) and `[:top_k]` keeps the best few.
- `if scores[i] >= min_score` is the **similarity threshold** (a bonus item). Anything weaker than MIN_SCORE is dropped. If nothing survives, the list is empty, and answer() uses that to say "not found".
- Each result is the original record plus its score, so you can print `source`, `chunk_id` and `score` while debugging.

Tuning MIN_SCORE. 0.25 is a starting point, not a law. Run a few questions with `min_score=0` and print the scores: good matches usually score clearly higher than unrelated ones. Set the threshold between the two groups on your own data.

Task 5 and Task 6: answer() (15 + 10 marks)

```
SYSTEM_PROMPT = (
    "You are the ShopKart shopping assistant. "
    "Answer ONLY using the context provided by the user message. "
    "Never use outside knowledge and never guess prices, specs or policies. "
    "If the context does not contain the answer, reply exactly with: "
    + NOT_FOUND +
    " Keep answers short and clear. For comparisons, mention both products."
)


def answer(question):
    hits = retrieve(question)

    # Task 6, layer 1: nothing similar enough was retrieved
    if not hits:
        return {"answer": NOT_FOUND, "sources": []}

    context = "\n\n".join(f"[Source: {h['source']}]\n{h['text']}" for h in hits)
    reply = get_client().chat.completions.create(
        model=LLM_MODEL,
        temperature=0,
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {question}"},
        ],
    )
    text = reply.choices[0].message.content.strip()

    # Task 6, layer 2: the LLM itself said the context has no answer
    if text.startswith("I'm sorry, I couldn't find"):
        return {"answer": text, "sources": []}

    sources = sorted({h["source"] for h in hits})
    return {"answer": text, "sources": sources}


def ask(question):
    result = answer(question)
    print(f"\nQ: {question}")
    print(f"A: {result['answer']}")
    if result["sources"]:
        print("Sources: " + ", ".join(result["sources"]))
```

The prompt is what makes the answer grounded.

- **System prompt:** tells the model it may use only the supplied context, must never guess prices or specs, and must reply with one exact sentence when the answer is missing. A fixed sentence is also easy for the code to detect.
- **Context block:** each retrieved chunk is labelled `[Source: products.txt]`, so the model can see where each fact came from.
- **temperature=0** makes the model stick to the text instead of being creative. For a factual shop assistant you want repeatable answers.
- **Sources:** collected from the chunks that were sent to the model, de-duplicated and sorted, and printed under the answer by ask() (source citations, 10 marks).

Task 6 uses two safety nets, because one is not enough:

Layer	Where	What it catches
1. Threshold	retrieve() returns nothing	Questions with no similar chunk at all. The LLM is never called.

Layer	Where	What it catches
2. Prompt rule	System prompt + startswith check	Questions that look similar to the catalog (for example "price of iPhone 17 Pro" shares the idea of a phone price) but whose answer is not in the retrieved text. The LLM says it cannot find it, and we hide the sources because nothing was actually used.

Run section: chat loop and the 23 test questions

```
if __name__ == "__main__":
    if "--test" in sys.argv:
        for question in TEST_QUESTIONS:
            ask(question)
    else:
        print("ShopKart assistant ready. Type 'quit' to exit.")
        while True:
            q = input("\nYour question: ").strip()
            if q.lower() in {"", "quit", "exit"}:
                break
            ask(q)
```

`TEST_QUESTIONS` (in the file) holds all 23 questions from the assignment. `--test` runs them in order; with no flag you get an interactive chat. Output for each question looks like this (example):

```
Q: What is the price of Samsung Galaxy M35?
A: The Samsung Galaxy M35 costs 19,999 INR.
Sources: products.txt
```

5. Model answers for the 23 test questions

What a correct, grounded answer looks like, and which chunk holds the information. Wording from your LLM will differ; the facts must match. "#n" is the chunk number inside that file.

Product questions

#	Question	Model answer	Found in
1	What is the price of Samsung Galaxy M35?	19,999 INR.	products #0
2	Which smartphones have 8GB RAM?	Samsung Galaxy M35 and Redmi Note 14.	products #0, #1
3	Compare Samsung Galaxy M35 and Redmi Note 14.	Both: 8GB RAM, 128GB storage, in stock, 1 year warranty. Samsung: 19,999 INR, 6000mAh, Super AMOLED. Redmi: 17,999 INR, 5500mAh, AMOLED.	products #0, #1
4	Which products are currently out of stock?	boAt Airdopes 141 (SK104).	products #2
5	Which laptop has 16GB RAM?	HP Pavilion Laptop (16GB RAM, 512GB SSD, Core i5, 59,999 INR).	products #1, #2
6	What is the battery capacity of Samsung Galaxy M35?	6000mAh.	products #0

Delivery questions

#	Question	Model answer	Found in
7	How many days does standard delivery take?	3 to 5 business days after the order is shipped.	delivery #0
8	Is express delivery available?	Yes, in metro cities only: 1 to 2 business days, extra 99 INR.	delivery #0, #1
9	How can I track my order?	A tracking link is sent by SMS and email after shipping; also under My Orders on the website or app.	delivery #1
10	What happens if my delivery is delayed?	If more than 2 business days late, contact support. If delayed more than 7 days, cancel for a full refund. Late express orders get the express fee back.	delivery #3, #4

Return and refund questions

#	Question	Model answer	Found in
11	What is the return window?	7 days from delivery.	return #0
12	How long does a refund take?	5 to 7 business days after the returned product passes quality check.	return #1
13	Can I return a damaged product?	Yes. Report it within 48 hours under My Orders with photos or an unboxing video; free pickup, then replacement or full refund.	return #2
14	Which products cannot be returned?	Opened earbuds and other in-ear audio, innerwear, gift cards, software and digital products, and items with a missing or tampered serial number.	return #3

Payment questions

#	Question	Model answer	Found in
15	Does ShopKart support UPI?	Yes: Google Pay, PhonePe, Paytm and BHIM; confirmed instantly.	payment #0
16	Is cash on delivery available?	Yes, for orders up to 10,000 INR in serviceable PIN codes, with a 30 INR fee.	payment #1
17	What happens if a payment fails?	If money was deducted but the order is not confirmed, it is refunded automatically in 5 to 7 business days. You can retry, and contact support if no refund after 7 days.	payment #2

Advanced reasoning questions

#	Question	Model answer	Found in
18	Which smartphone is cheaper, Samsung or Redmi?	Redmi Note 14 (17,999 INR) is 2,000 INR cheaper than Samsung Galaxy M35 (19,999 INR).	products #0, #1
19	Find an in-stock product under Rs. 20,000.	Redmi Note 14 (17,999 INR) and Samsung Galaxy M35 (19,999 INR). boAt Airdopes is under 20,000 but out of stock, so it does not qualify.	products #0, #1, #2
20	Recommend a smartphone with 8GB RAM and explain why.	Both have 8GB. Redmi Note 14 for the lower price (17,999 INR); Samsung Galaxy M35 for the bigger battery (6000 vs 5500mAh) and Super AMOLED display. Reasons must come from the catalog only.	products #0, #1

Hallucination tests (expected: not found)

#	Question	Model answer	Found in
21	What is the price of iPhone 17 Pro?	Says it cannot find this in the knowledge base.	none
22	Does ShopKart sell refrigerators?	Says it cannot find this in the knowledge base.	none
23	What is the CEO's phone number?	Says it cannot find this in the knowledge base.	none

Checking hallucinations. A wrong answer to 21 to 23 is any invented price, product, or phone number. If your bot makes one up, first lower the risk in the prompt (repeat the "only use the context" rule), then raise MIN_SCORE a little, and re-run all three.

6. Bonus: FAISS, better chunking, next steps

Upgrade to FAISS (vector database)

NumPy compares the question against every chunk. That is fine for 17 chunks, but slow for millions. FAISS is a library built for fast nearest-neighbour search. `IndexFlatIP` uses inner product, which equals cosine similarity for normalized vectors, so the scores stay identical to the NumPy version.

```
# pip install faiss-cpu
import faiss

index = faiss.IndexFlatIP(vectors.shape[1])      # 384 dimensions
index.add(vectors.astype("float32"))             # FAISS needs float32


def retrieve(question, top_k=TOP_K, min_score=MIN_SCORE):
    q = embedder.encode([question], normalize_embeddings=True).astype("float32")
    scores, ids = index.search(q, top_k)         # shapes: (1, top_k)
    return [
        {**records[i], "score": float(sc)}
        for sc, i in zip(scores[0], ids[0])
        if i != -1 and sc >= min_score           # -1 means "no result"
    ]
```

Replace the NumPy `retrieve()` with this one; `answer()` does not change. This is why vector databases matter: the rest of the program does not care how search is done.

Better chunking for the product catalog

One chunk per product guarantees a product's name, price and specs always stay together:

```
import re

def chunk_products(text):
    parts = re.split(r"\n(?=PRODUCT \d+)", text.strip())   # split before each "PRODUCT n"
    return [p.strip() for p in parts if p.strip().startswith("PRODUCT")]
```

Use it for `products.txt` and keep word-based chunk() for the policy files.

Other bonus items

- **Category filter:** add a `category` field to each product record and keep only matching records before ranking, for example only "Smartphone".
- **Conversation history:** keep a list of past messages and include the last few in the `messages` list sent to Groq.
- **Streamlit UI:** wrap `answer()` with `st.chat_input` and `st.chat_message`.

- **PDF catalogs:** extract text with a library such as pypdf and feed it through the same chunk() function.

7. Common problems and fixes

Problem	Cause and fix
KeyError: 'GROQ_API_KEY'	The .env file is missing, misnamed, or not in the folder you run from. Create .env with GROQ_API_KEY=your_key and run from that folder.
FileNotFoundError: kb/products.txt	Run the script from the folder that contains the kb/ folder, and check the four filenames.
Model download is slow or fails	First run only. Check your internet connection; once downloaded the model is cached.
Groq says the model is unavailable	Model names change over time. Open the Groq console models list and set LLM_MODEL to a current model.
Answer is wrong but the facts exist	Print retrieve(question, min_score=0) and look at the scores and chunks. If the right chunk is not in the top 4, raise TOP_K or fix the chunking.
Bot invents answers for questions 21 to 23	Strengthen the system prompt and raise MIN_SCORE slightly.
Everything returns "not found"	MIN_SCORE is too high. Print the scores and lower it.

8. Where each mark is earned

Criteria	Marks	Where in this solution
Knowledge base creation	10	Four kb/ files with every required topic (Section 3)
Multiple document loading	10	load_documents()
Chunking	10	chunk() with overlap, build_records() with filename metadata
Embedding generation	15	SentenceTransformer all-MiniLM-L6-v2 with normalize_embeddings=True
Semantic retrieval	20	retrieve(): cosine similarity with NumPy, top-k, threshold
LLM integration	15	answer(): system prompt, context, Groq call with temperature 0
Hallucination handling	10	Threshold plus prompt rule; fixed "not found" sentence
Source citations	10	[Source: ...] labels in the context and the Sources line in ask()
Total	**100**	